

CSC3066

Deep Learning

Different Activation and Loss Functions

Anna Jurek-Loughrey

1 February 2023



Outline

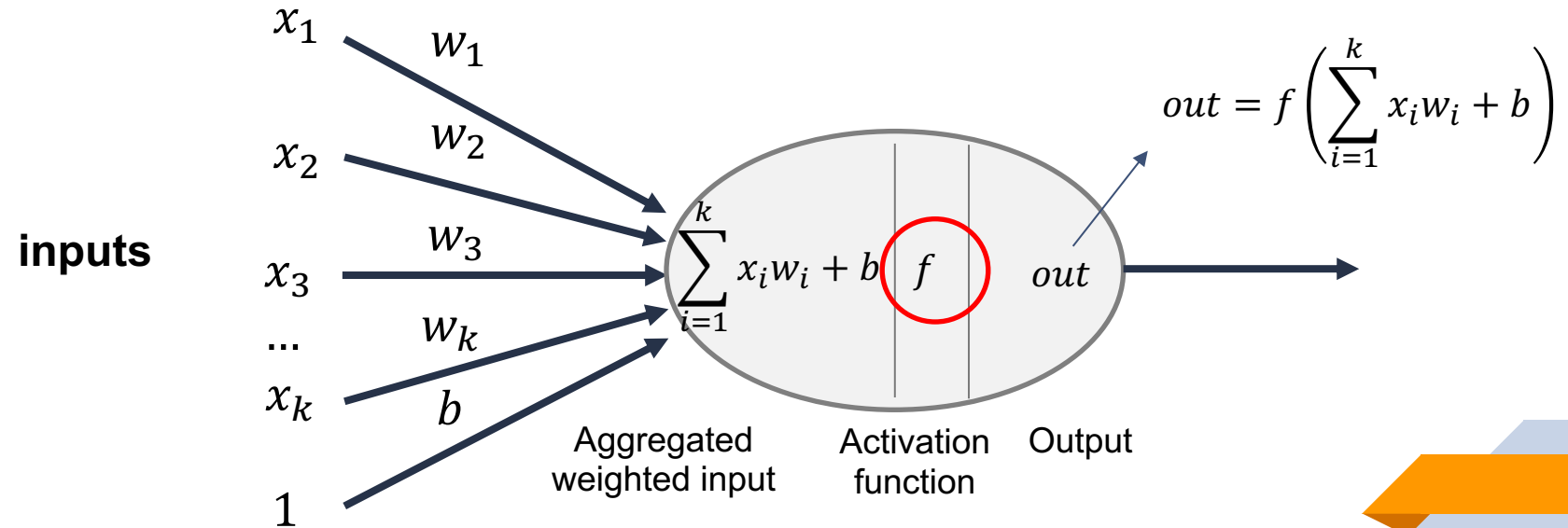
- ❑ Overview of different activation functions
- ❑ Overview of different error/loss functions

Neural Network:

Activation Functions

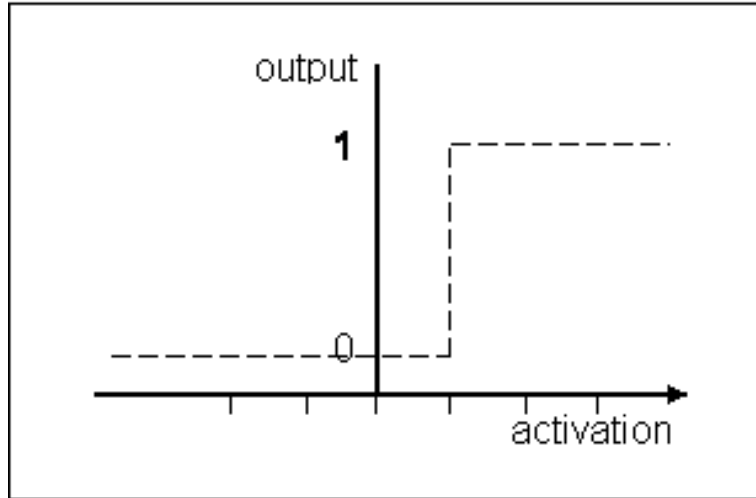
Activation Function: Recap

- Activation function is a mathematical equation attached to a neuron that determine its output.



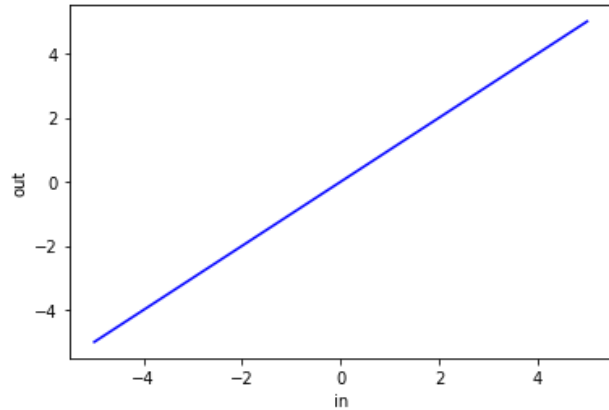
Common Activation Functions: Binary Step Function

- ❑ Problem: it does not allow multi-value outputs. It does not support multi class classification problem. The gradient of the step function is zero.



$$f(x) = \begin{cases} 0 & \text{if } x \leq \text{threshold} \\ 1 & \text{if } x > \text{threshold} \end{cases}$$

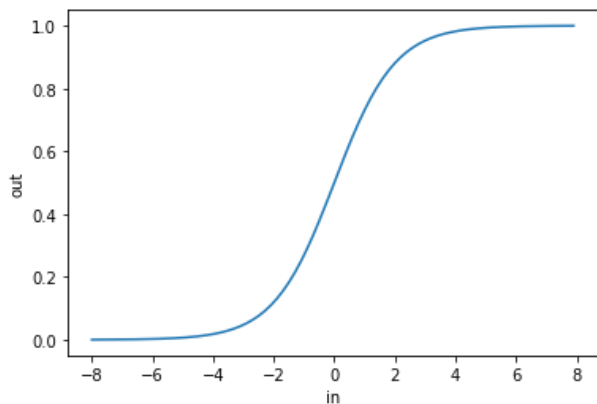
Common Activation Functions: Identity/Linear



$$f(x) = ax$$

- ❑ A straight line function: a is a constant value.
- ❑ The linear function alone doesn't capture complex patterns.
- ❑ Gradient of the linear function is a constant and has no relation to the input.

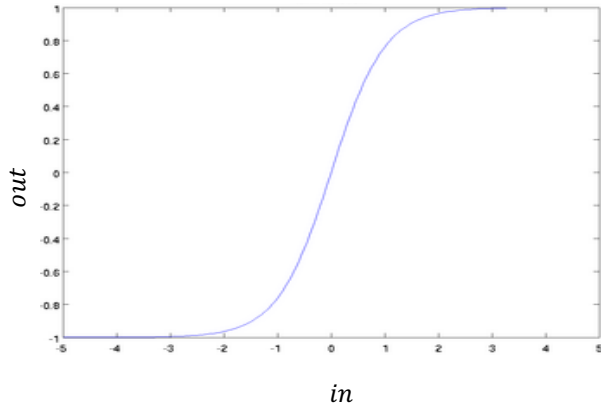
Common Activation Functions: Sigmoid/Logistic Function



$$f(x) = \frac{1}{1 + e^{-x}}$$

- ❑ A non-linear function so can capture more complex patterns
- ❑ Continuously differentiable
- ❑ Output values are bounded between 0 and 1 so don't get too large
- ❑ Useful for models where we have to predict the probability as an output (values between 0 and 1).
- ❑ Can suffer from “vanishing gradient”

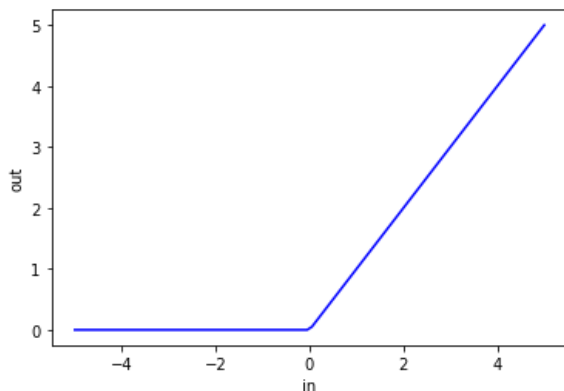
Common Activation Functions: Tanh Function



$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ❑ A non-linear function so can capture more complex patterns
- ❑ Continuously differentiable
- ❑ Output values are bounded between -1 and 1 so don't get too large
- ❑ Can suffer from “vanishing gradient”

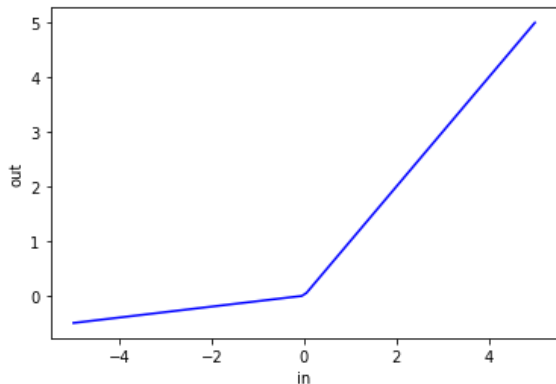
Common Activation Functions: ReLU (Rectified Linear Unit)



$$f(x) = \max(0, x)$$

- ❑ Fast to compute
- ❑ It does not activate all the neurons at the same time, which makes it more computationally efficient.
- ❑ Reduces the vanishing gradient problem.
- ❑ The range of ReLu is $[0, \infty)$, which means it can blow up the activation.
- ❑ “dying ReLU problem”: the weights and biases for some neurons may never be updated.

Common Activation Functions: Leaky ReLU



$$f(x) = \max(0.01 \times x, x)$$

- ❑ Attempts to solve the “dying ReLU problem”
- ❑ Alternatively, instead of using 0.01, this can also be a parameter, which is then learned during training alongside the weights. This is referred to as Parametric ReLU (PReLU):

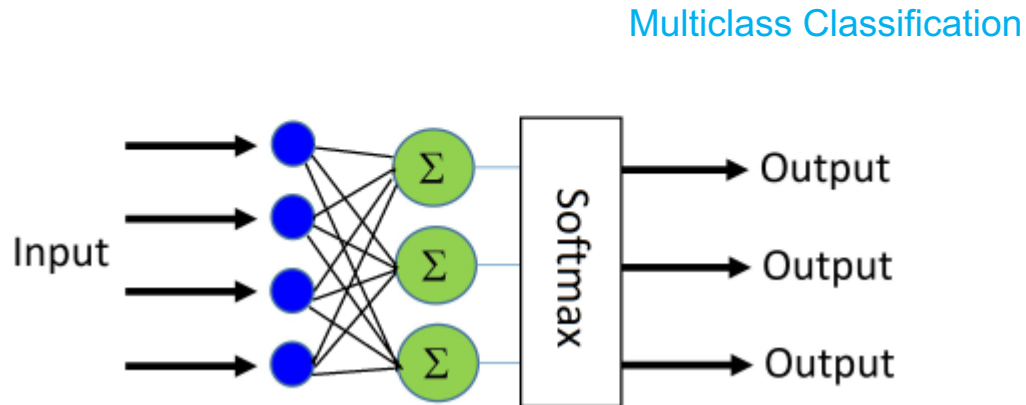
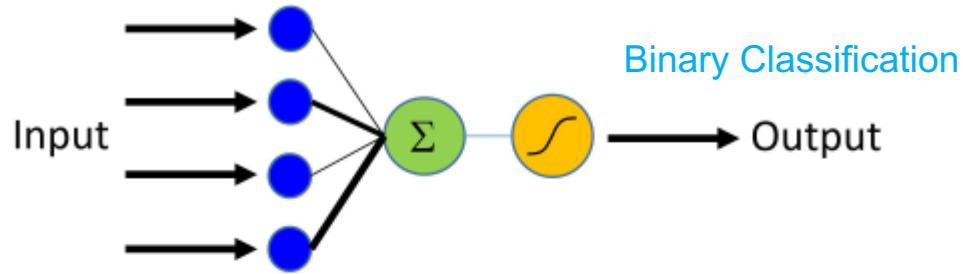
Common Activation Functions: Softmax

$$\begin{pmatrix} -0.3 \\ 1.4 \\ 2.6 \end{pmatrix} \rightarrow \text{softmax} \rightarrow \begin{pmatrix} 0.0406 \\ 0.2221 \\ 0.7373 \end{pmatrix}$$

$$p_i = \frac{e^{x_i}}{\sum_{k=1}^c e^{x_k}}$$

- ❑ Softmax is used as the output function of the last layer in neural networks
- ❑ c represents the number of classes
- ❑ Each value ranges between 0 and 1 and the sum of all values is 1 so can be used to model probability distributions
- ❑ Only used in the output layer rather than throughout the network

Common Activation Functions: Softmax



Common Activation Functions: Derivatives

Name	Equation	Derivative
Binary Step	$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Identity	$f(x) = x$	$f'(x) = 1$
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$
Leaky ReLU	$f(x) = \begin{cases} \alpha x & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$
Softmax	$p_i = \frac{e^{x_i}}{\sum_{k=1}^c e^{x_k}}$	$\frac{\partial p_i}{\partial x_j} = \begin{cases} p_i(1 - p_j) & \text{if } i = j \\ -p_i p_j & \text{otherwise} \end{cases}$

Neural Network:

Objective/Loss/Error Function



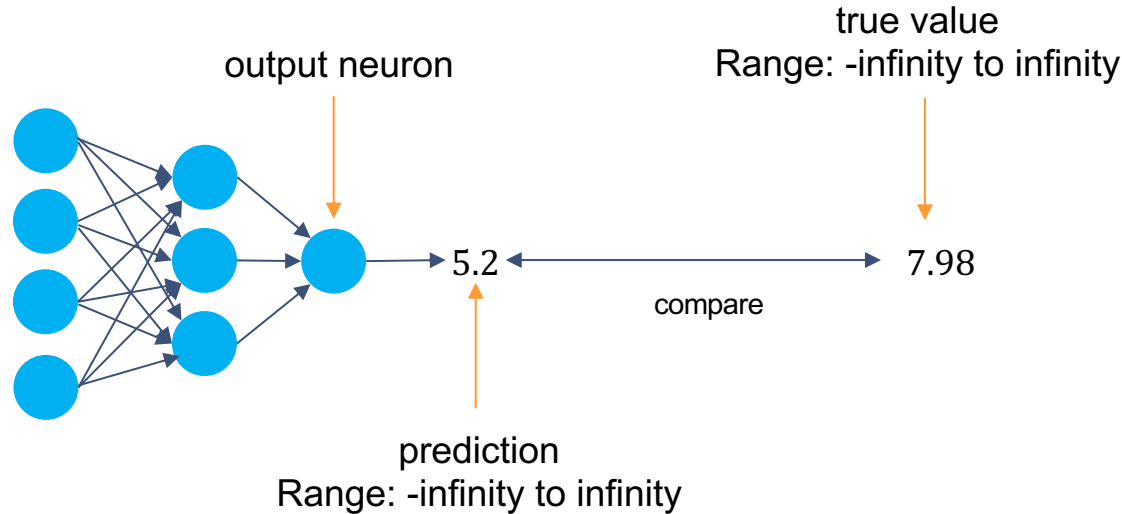
Artificial Neural Network: Loss/Objective/Error functions

- ❑ The loss function allows us to evaluate the success of our Neural Network
- ❑ Depending on the problem, we may need to evaluate the performance of our NN differently
- ❑ What problem are we trying to solve?
 - To predict a numerical value? → regression
 - To predict a categorical value? → classification
 - How many categories? → binary vs multi-class classification



Regression - Predicting a numerical value

- The final layer of the neural network has one neuron, which returns a continuous numerical value.





Regression - Predicting a numerical value

- Suitable Activation Functions for the Output Neuron

- Linear: $f(x) = x$

- ReLU: $f(x) = \max(0, x)$

- Leaky ReLU: $f(x) = \max(0.01 \times x, x)$

- Sigmoid / Tanh – if values are between 0 and 1 / -1 and 1



Regression - Predicting a numerical value

❑ Error/Loss Function

Squared Error – finds the average squared difference between the predicted value and the true value

$$L = \frac{1}{2}(y - out)^2$$

where y is the true value and *out* is the predicted value.



Regression - Predicting a numerical value

□ Derivative of the squared error loss function

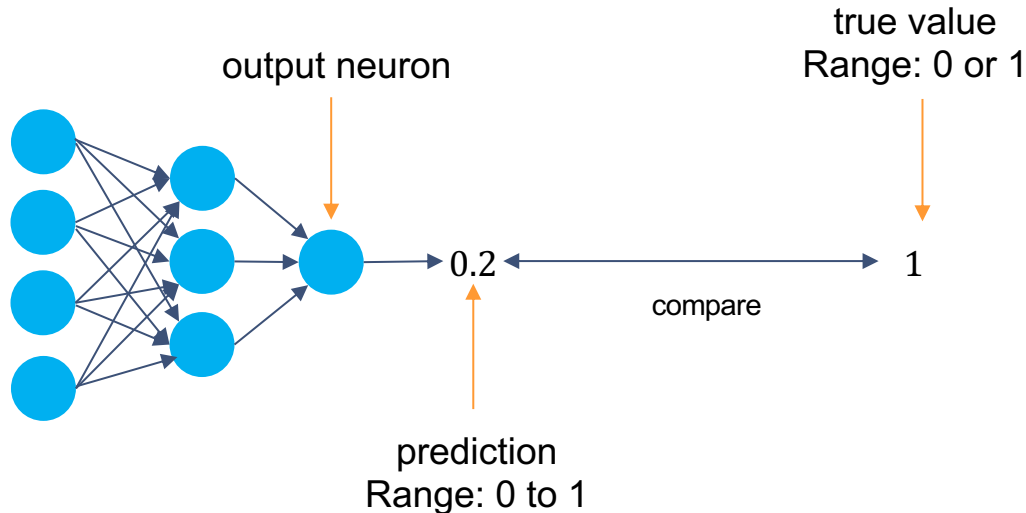
$$E = \frac{1}{2}(y - out)^2$$

$$\frac{\partial E}{\partial out} = out - y$$



Classification: Predicting a binary outcome

- The final layer of the neural network has one neuron, which returns a value between 0 and 1, which can be inferred as a probability.

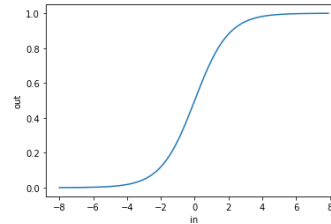




Classification: Predicting a binary outcome

□ Suitable Activation Functions for the Output Neuron

□ Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$





Classification: Predicting a binary outcome

❑ Error/Loss Function

(Binary) Cross Entropy – measure of the difference between two probability distributions (binary classification case):

$$E = -(y \log(out) + (1 - y) \log(1 - out))$$

where y is the true value and *out* is the predicted value



Classification: Predicting a binary outcome

□ Derivative of the binary cross-entropy loss function

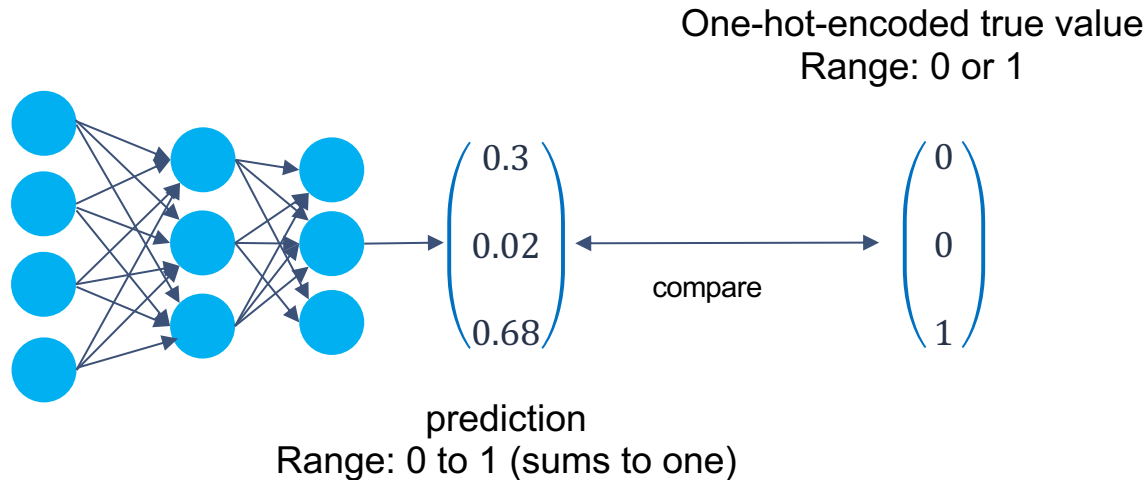
$$E = -(y \log(out) + (1 - y) \log(1 - out))$$

$$\frac{\partial E}{\partial out} = -\frac{y}{out} + \frac{1 - y}{1 - out}$$



Classification: Predicting a single label from multiple classes

- ❑ The final layer of the neural network has one neuron for each of the classes, which returns a value between 0 and 1.
- ❑ The output can be considered as probability distribution.





Classification: Predicting a single label from multiple classes

- Suitable Activation Functions for the Output Neuron

- Softmax: $p_i = \frac{e^{x_i}}{\sum_{k=1}^c e^{x_k}}$



Classification: Predicting a single label from multiple classes

□ Error/Loss Function

Cross Entropy – quantifies the difference between two probability distribution

$$E = - \sum_{i=1}^n y_i \log out_i$$

Where n is the number of classes, y is the true value (binary indicator 0 or 1) and out is the predicted probability for class i .



Classification: Predicting a single label from multiple classes

- Derivative of the cross-entropy loss function for the softmax function
 - The derivative $\frac{\partial E}{\partial out_i} \times \frac{\partial out_i}{\partial in_i}$ of the loss function with respect to the softmax input in_i can be calculated as: $out_i - y_i$
 - The detailed derivation of cross-entropy loss function with softmax activation function can be found at <https://deepnotes.io/softmax-crossentropy>

Next to come:

Training Multilayer Perceptron